KRIPYA TECHNICAL EVALUATION · INFRASTRUCTURE MANAGEMENT TOOL

Infra Portal

A self-service portal for requesting servers on Proxmox VE and AWS

Muthu · November 2026 · github.com/Muthujar/infra-portal

THE PROBLEM

Getting a server today means a ticket and a wait

- Developers raise a ticket and wait for the DevOps team.
- Servers are built by hand, so they differ.
- There is no clear record of who approved what.
- Forgotten test servers keep using capacity.

Goal: a catalog, an approval step and automation, like VMware vRealize Automation (Aria).

THE SOLUTION

Four steps from request to ready server

1 · DEVELOPER

Request

Picks a server, environment and lease.

2 · API

Check

Validates the input and team quota.

3 · TEAM LEAD

Approve

Approves it. PROD needs an admin. Terraform is generated.

4 · WORKER

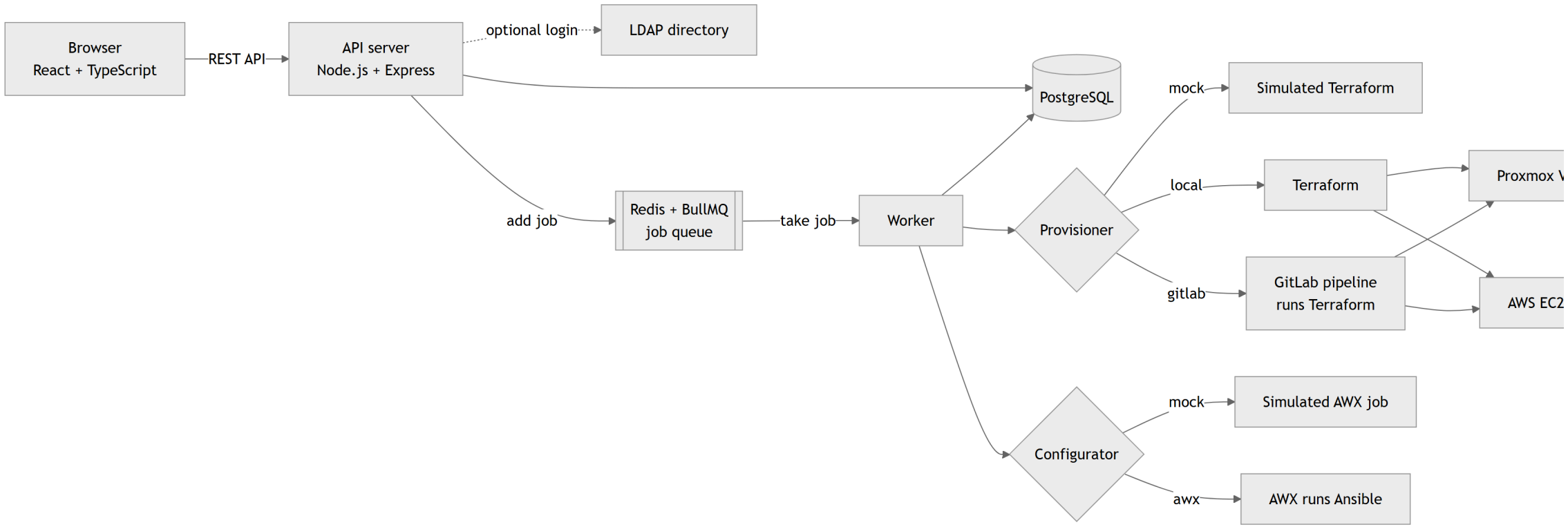
Build

Runs Terraform, then configures the server.

Every change is in the audit log. Servers are deleted when their lease ends.

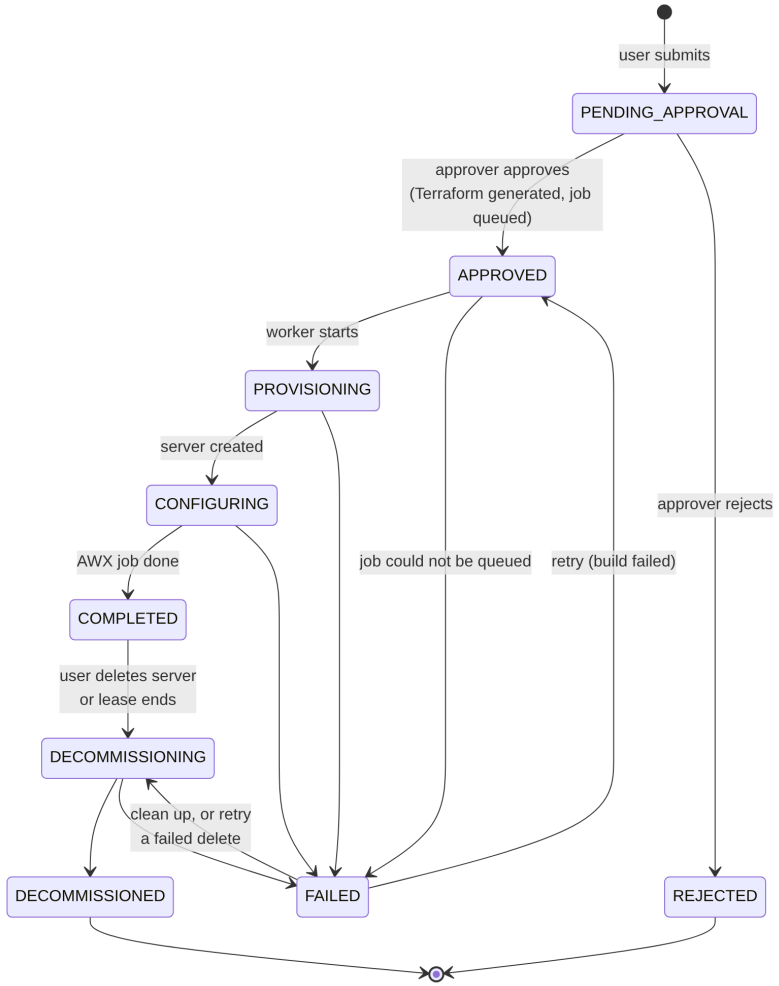
ARCHITECTURE

A queue and worker handle the slow work



REQUEST LIFE CYCLE

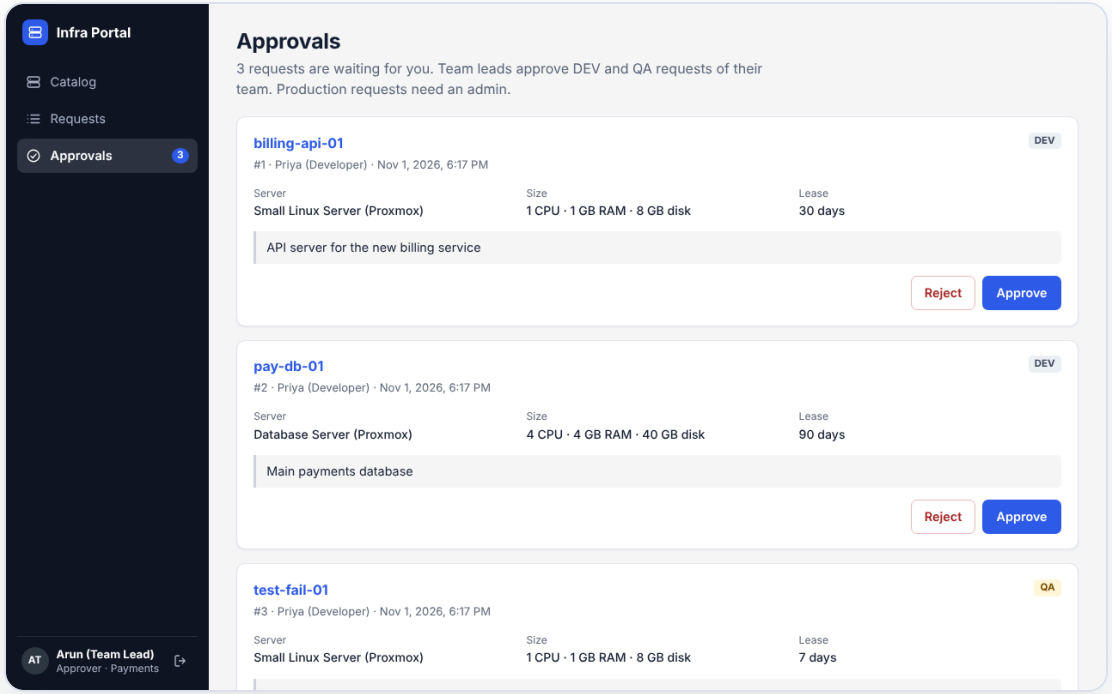
One state machine controls every status change



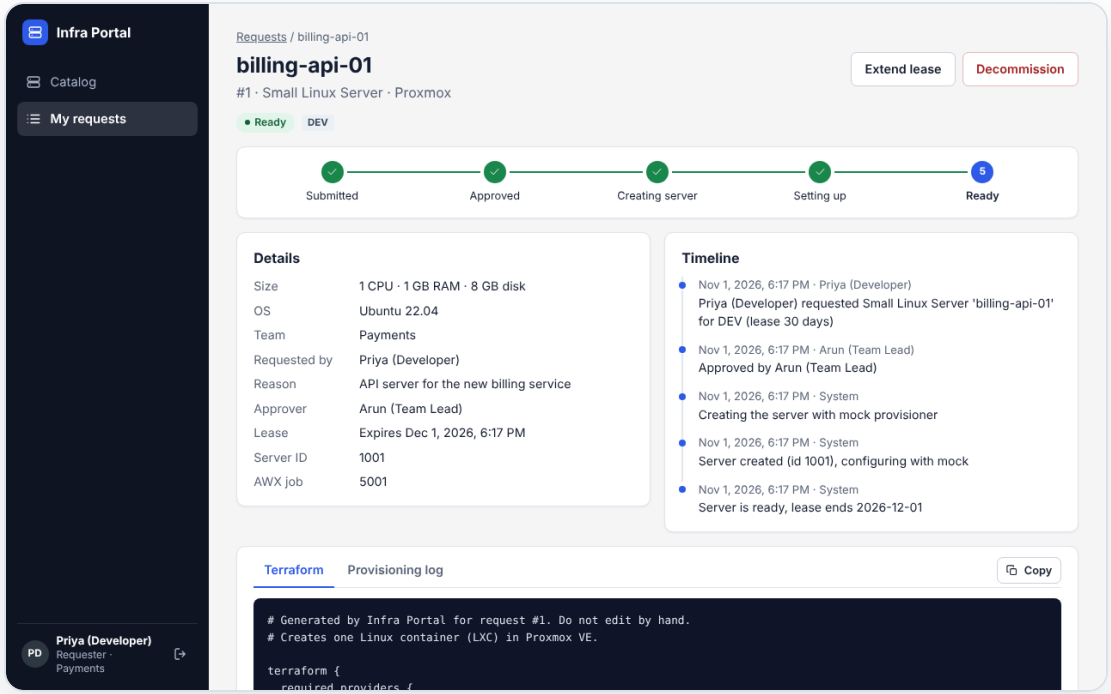
- Only allowed moves are accepted.
- Two people cannot act on a request at once.
- Every change is written to the audit log.

DEMO

From approval to a ready server



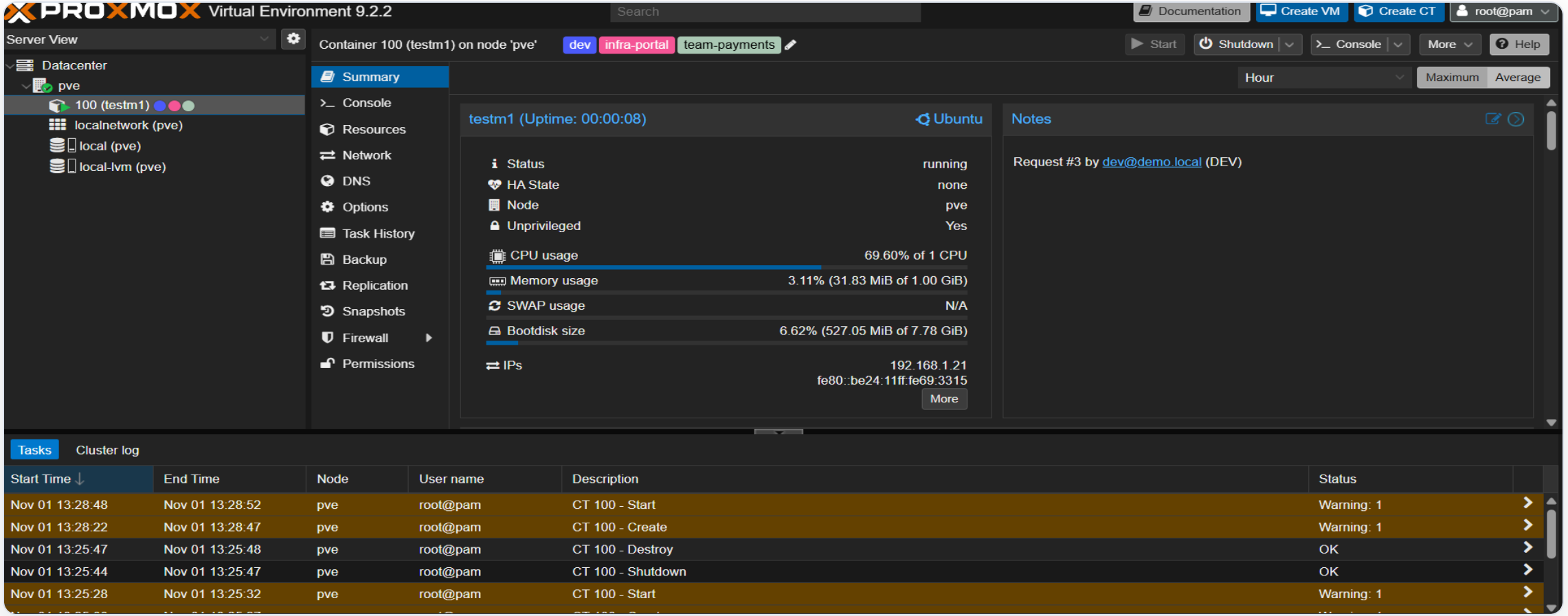
Team lead approves



Live status, timeline and Terraform

TESTED FOR REAL

A real container on Proxmox VE 9.2



Created by Terraform in about 40 seconds, then deleted on decommission.

CONTROLS

Who can do what, and how much

Roles
Requester, approver, admin. PROD needs an admin.

Quotas
CPU, RAM and disk limits for each team.

Audit log
Who did what, and when.

Leases
Expired servers are deleted automatically.

Secrets
Never saved in Terraform files or the database.

LDAP login
Directory groups set role and team.

SCOPE

What is real and what is not

PART	STATUS
Portal, workflow, roles, quotas, leases, audit log	Done
Terraform generation for Proxmox and AWS	Done
Provisioning on real Proxmox VE	Tested
LDAP login	Tested
AWS EC2	Validated, not applied
GitLab CI	Commits tested, pipeline not run
AWX	Mocked in the demo

QUALITY AND WHAT COMES NEXT

Tested on every push, ready to grow

- **44 automated tests**
- CI checks the backend, frontend, Terraform and Docker
- A smoke test takes a request to Ready

- **If this goes forward**
- Run the GitLab pipeline on a verified account
- Install AWX, apply AWS with remote state
- Secrets in a vault
- Email or Slack notifications